Contents

#pragma once #include "component.h" #include "saveable.h" #include <glm/glm.hpp> #include <glm/ext.hpp> #include <vector> #include <string> #include "../dflib/dflib.h"
namespace engine {
class Component;
class Transform : public Saveable { public:
Transform();
/**

- "load" constructor
-
- Called by the level loader to deserialize transforms from json
- Simply reads the corresponding fields from the json file and populates the class fields
-
- @param value The JSON to deserialize

*/ Transform(json value);
/**

- Same as default constructor, but with the node name set
-
- @param name The desired node name

*/ Transform(std::string name);
/**

- Destroys this transform, all of its components, and then recursively destroys
- its children

*/ ~Transform();
/**

- Sets the node's name

-
- @param name The desired node name

*/ void SetName(std::string name);
/**
- Save's the transform's properties to a JSON object
-
- @returns the JSON object representing the transform

*/ json Save() override;
/**
- Load the transform properties saved with 'Save()' into the transform
-
- @returns true on success, false on failure

*/ bool Load(json value) override;
/**
- This is the render function called by the editor

*/ void ProcessEditorRender();
/**
- This is the render function called by the game

*/ void ProcessRender();
/**
- Runs once per frame when the game simulation is running

*/ void ProcessTick(float dt);
/**
- This is the initialization function called to initialize
- components.

*/ void ProcessInit();
/**

- Add a component to this transform
-
- @param component A pointer to the component to add

/ **void AddComponent(Component** component);

/**
- Get a component on this transform by name
-
- If multiple components have the same name, the first
- in the array is returned
-
- If no component has the given name, return nullptr
-
- @param name The name of the component to return
-
- @returns A pointer to the component with the given name, or nullptr

/ **Component** GetComponent(std::string name);

/**
- Add a child transform to this transform
-
- If the transform is already a child of this transform
- log a warning and ignore it
-
- @param tr the transform to add as a child
-

*/ void AddChild(Transform *tr);

/**

- Return the child transform that has the given id
-
- If no child has the given ID, return nullptr
-
- @param id The id of the transform to return
-
- @returns A pointer to the transform with the given id, or nullptr

*/ Transform *FindTransformByUID(uint64$_t$ id);
/**
- Return the attached component that has the given id
-
- If no component has the given ID, return nullptr
-
- @param id The id of the component to return
-
- @returns A pointer to the component with the given id, or nullptr

/ **Component** FindComponentByUID(uint64$_t$ id);
/**
- Return the child transform with the given name
-
- If more than one child has the given name, return the first in the array
-
- If no child has the given name, return nullptr
-
- @param name The name of the child to return

-
- @returns A pointer to the transform with the given name, or nullptr

*/ Transform *FindChildByName(std::string name);
/**
- Removes the given transform from the children array, and returns it
-
- Does not automatically destroy the transform
-
- If the given transform does not exist in the child array, return nullptr
-
- @param tr The transform to remove
-
- @returns The transform that was removed

/ **Transform** RemoveChild(Transform *tr);
/**
- Return the children array of this transform
-
- @returns The children array of this transform

/ std::vector<Transform> GetChildren();
/**
- Translates this transform in local coordinates
-
- @param offset The vector to translate the transform by

*/ void Translate(glm::vec3 offset);
/**
- Scales this transform by the given multiplier

-
- @param amount The vector to multiply the scale by

*/ void Scale(glm::vec3 amount);
/**
- Rotates the transform along the given axis by the given amount
-
- @param axis The axis to rotate around
- @param amount The rotation amount, in degrees

*/ void Rotate(glm::vec3 axis, float amount);
/**
- Returns the forward direction of this transform
-
- @returns the forward direction of the transform

*/ glm::vec3 Forward();
/**
- Returns the backward direction of this transform
-
- @returns the backward direction of the transform

*/ glm::vec3 Backward();
/**
- Returns the up direction of this transform
-
- @returns the up direction of the transform

*/ glm::vec3 Up();
/**
- Returns the down direction of this transform
-

- @returns the down direction of the transform

*/ glm::vec3 Down();
/**

- Returns the left direction of this transform
-
- @returns the left direction of the transform

*/ glm::vec3 Left();
/**

- Returns the right direction of this transform
-
- @returns the right direction of the transform

*/ glm::vec3 Right();
/**

- Calculate the model matrix given the instances position, rotation, and scale
-
- @returns the model matrix corresponding to this instance

*/ glm::mat4 GetModelMatrix();
/**

- Calculates the model matrix with the scale normalized to 1
-
- This is used internally for some editor functionality.
-
- @returns The model matrix corresponding to this instance with a unit scale

*/ glm::mat4 GetModelMatrixUnscaled();
/**

- Calculates the position of this transform in global coordinates
-
- @returns the global position vector of this transform
-

*/ glm::vec3 GetGlobalPosition();
/**

- Returns the position of this transform in local coordinates
-
- @returns the local position vector of this transform

*/ glm::vec3 GetLocalPosition();
/**

- Returns the rotation of this transform in global coordinates
-
- @returns the quaternion rotation of this transform in global coordinates

*/ glm::quat GetGlobalRotation();
/**

- Returns the rotation of this transform in local coordinates
-
- @returns the quaternion rotation of this transform in local coordinates

*/ glm::quat GetLocalRotation();
/**

- Returns the scale of this transform in global coordinates
-
- @returns the scale value of this transform in global coordinates

*/ glm::vec3 GetGlobalScale();
/**

- Returns the scale of this transform in local coordinates
-
- @returns the scale value of this transform in local coordinates

*/ glm::vec3 GetLocalScale();
/**

- Returns the name of this transform
-
- @returns the name of this transform

*/ std::string GetName();
/**

- Returns the uid of this transform
-
- @returns the uid of this transform

*/ uint64$_t$ GetUID();
/**

- Draws the editor UI for this transform

*/ void DrawWidget();
glm::vec3 position; glm::quat rotation; glm::vec3 scale; std::string name;
uint64$_t$ uid; glm::vec3 rotationEuler; //slaved to the quaternion
Transform *parent = nullptr; std::vector<Transform *> children; std::vector<Component*>
components;
private: };
}